

PRAKTIKUM SISTEM OPERASI

MODUL XI
MEMORI XINU



Oleh:

Marvel Sanjaya Setiawan

2311104053

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

JURNAL

1. Freememory command.

Jawaban:



```
Activities gedit Mon 11 May, 05:28
xsh_freememory.c
~/xinu/shell

/* Nama : Marvel Sanjaya Setiawan */
/* NIM : 2311104053 */

#include <xinu.h>

shellcmd xsh_freememory(int nargs, char *args[])
{
    struct memblk *memptr;
    uint32 totalFreeSpace = 0;

    printf("Free List:\n");
    printf("%-14s %-14s %-14s\n", "Block address", "Length (dec)", "Length (hex)");
    printf("%-14s %-14s %-14s\n", "-----", "-----", "-----");

    for (memptr = memlist.mnext; memptr != NULL; memptr = memptr->mnext){
        printf(" 0x%08x %10d 0x%08x\n",
            (uint32)memptr,
            (int32)memptr->mlength,
            memptr->mlength);
        totalFreeSpace += memptr->mlength;
    }

    printf("Total FreeSpace %10u\n", totalFreeSpace);
    return OK;
}
```

a. Buat file source code xsh_freememory.c

Masuk ke ~/xinu/shell, buat xsh_freememory.c, isi dengan fungsi xsh_freememory, lalu simpan.

```
1. /* Nama : Marvel Sanjaya Setiawan */
2. /* NIM : 2311104053 */
3.
4. #include <xinu.h>
5.
6. shellcmd xsh_freememory(int nargs, char *args[])
7. {
8.     struct memblk *memptr;
9.     uint32 totalFreeSpace = 0;
10.
11.     printf("Free List:\n");
```

```

12.     printf("%-14s %-14s %-14s\n", "Block address", "Length (dec)", "Length (hex)");
13.     printf("%-14s %-14s %-14s\n", "-----", "-----", "-----");
14.
15.     for (memptr = memlist.mnext; memptr != NULL; memptr = memptr->mnext) {
16.         printf("    0x%08x    %10d    0x%08x\n",
17.             (uint32)memptr,
18.             (int32)memptr->mlength,
19.             memptr->mlength);
20.         totalFreeSpace += memptr->mlength;
21.     }
22.
23.     printf("Total FreeSpace %10u\n", totalFreeSpace);
24.     return OK;
25. }
26.

```

b. Daftarkan command di shell.c

Buka xinu/shell/shell.c, tambahkan {"freememory", xsh_freememory} ke daftar command, lalu simpan.

```

1. {"freememory", xsh_freememory},
2.

```

c. Tambahkan deklarasi di shprototypes.h

Buka ../include/shprototypes.h, tambahkan extern shellcmd xsh_freememory(int32, char *[]);, lalu simpan.

```

1. extern shellcmd xsh_freememory(int, char *[]);
2.

```

d. Compile

Masuk ke ~/xinu/compile, jalankan make clean && make, tunggu hingga selesai tanpa error.

e. Testing

Jalankan make run, tunggu prompt xsh \$, lalu ketik freememory dan cek output daftar free memory block.

```
Activities Terminal Mon 11 May, 05:46
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
[0x0011FF40 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

-----
XINU
-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018e110 -975128 0xffff1ee8
0x00100000 5070848 0x004d6000
0x005e8000 57344 0x0000e000
Total FreeSpace 4153064
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

2. Jawaban Teori.

- Mengapa Xinu memisahkan data segment dan BSS segment?
- Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
- Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
- Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
- Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban:

- Data segment menyimpan variabel global dan statis yang sudah diinisialisasi sehingga nilainya harus disimpan di file binary. BSS segment menyimpan variabel yang belum diinisialisasi dengan nilai default 0, sehingga tidak perlu menyimpan data di binary dan cukup dicatat ukurannya saja. Pemisahan ini menghemat ukuran file binary secara signifikan.
- Saat alokasi menggunakan getmem, blok diambil dari free list sehingga ukuran free space berkurang. Saat dealokasi menggunakan freemem,

blok dikembalikan ke free list sehingga free space bertambah. Jika blok yang dikembalikan bersebelahan dengan blok bebas lain, Xinu akan menggabungkannya atau coalescing untuk mengurangi fragmentasi.

- c. Stack dikelola otomatis oleh hardware dan compiler sehingga memori bebas otomatis saat fungsi selesai. Heap dikelola secara manual sehingga rawan memory leak, dangling pointer, double free, dan fragmentasi. Di Xinu tidak ada garbage collector sehingga kesalahan heap bisa langsung merusak sistem.**
- d. Karena jumlah dan ukuran free block berubah secara dinamis. Linked list mudah tumbuh dan menyusut, mendukung penggabungan blok bersebelahan, dan overhead metadata sangat kecil karena pointer next disimpan langsung di dalam blok itu sendiri.**
- e. Tantangan utamanya adalah fragmentasi memori, tidak adanya proteksi memori antar proses, manajemen memori yang bersifat manual sehingga rawan memory leak, potensi race condition saat banyak proses mengakses free list secara bersamaan, serta tidak adanya mekanisme kompaksi memori untuk memperbaiki fragmentasi yang sudah terjadi.**